MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (Honours)/MGU-BBA (Honours)/MGU-BCA (Honours)

Special Examinations, July 2026 (for 2024 Admission Regular Examinations, October 2025) Third SEMESTER Discipline Specific Core Course

(DSC) - MG3DSCCSC201 - DATA STRUCTURES USING C (2024 ADMISSION ONWARDS)

ANSWER KEY

Duration: 1 Hour 30 Minutes **Maximum Marks: 50**

Remember(K), Understand(U), Apply(A), Analyse(An), Evaluate(E), Create(C), Skill(S), Interest(I) and Appreciation(Ap)

Students should attempt at least one question from each course outcome to enhance their overall outcome attainability.

Part A Very Short Answer Questions Answer all questions

Each question carries 2 marks [2x5 = 10]

1. What is a parameterized function?

Answer:
A **parameterized function** is a function that accepts one or more values (called **parameters**) as input. These parameters allow the function to perform operations using different data each time it is called.

Example:

```
def add(a, b):
    return a + b
```

Marking Scheme:

- Definition of parameterized function – **1 Mark**
- Example or mention that it accepts arguments – **1 Mark**

2. Discuss how does a stack help in postfix expression evaluation.

Answer:
A **stack** is used to evaluate postfix expressions by following these steps:

1. Scan the expression from left to right.
2. Push operands onto the stack.

3. When an operator is encountered, pop the required operands, perform the operation, and push the result back onto the stack.
4. The final value remaining in the stack is the result.

Marking Scheme:

- States that operands are pushed and operators pop operands – **1 Mark**
- Mentions the final result remains on the stack – **1 Mark**

3. Examine binary tree commonly used to implement heaps.

Answer:
A **heap** is implemented using a **complete binary tree**, where:

- All levels are completely filled except possibly the last level, which is filled from left to right.
- It satisfies the heap property (Max Heap or Min Heap).

Marking Scheme:

- Identifies heap as a complete binary tree – **1 Mark**
- Mentions complete filling and/or heap property – **1 Mark**

4. Explain why merge sort is considered a divide and conquer algorithm.

Answer:
Merge sort is called a **divide and conquer** algorithm because it:

1. **Divides** the array into two halves.
2. **Conquers** by recursively sorting each half.
3. **Combines** the sorted halves by merging them into a single sorted array.

Marking Scheme:

- Mentions dividing into smaller subproblems – **1 Mark**
- Mentions merging sorted subarrays – **1 Mark**

5. List the steps for deleting the first node of a singly linked list

Answer:
Steps:

1. Check whether the linked list is empty.

2. Store the first node in a temporary pointer.
3. Move the head pointer to the next node (head = head->next).
4. Delete (free) the temporary node.

Marking Scheme:

- Mentions updating the head pointer – **1 Mark**
- Mentions deleting/freeing the old first node (or lists the correct sequence) – **1 Mark**

Part B Short Answer Questions

Answer any 5 questions Each question carries 5 marks [5x5 = 25]

6 Interpret how the number of comparisons in linear search and binary search changes as the dataset size increases.

Answer:

As the **dataset size increases**, the number of comparisons required by **linear search** and **binary search** changes differently. As the **dataset size increases**, the number of comparisons required by **linear search** and **binary search** changes differently.

- **Linear Search**
    - Checks each element one by one until the target is found.
    - As the dataset grows, the number of comparisons increases **linearly** (proportional to **n**).
    - Worst-case time complexity: **O(n)**.
- **Binary Search**
    - Works only on a **sorted** dataset.
    - Divides the search space into two halves after each comparison.
    - As the dataset grows, the number of comparisons increases **very slowly** (logarithmically).
    - Worst-case time complexity: **$O(\log_2 n)$**.

For **small datasets**, both methods may perform similarly. However, as the dataset becomes **larger**, **binary search** requires **far fewer comparisons** than **linear search**, making it much more efficient for searching in sorted data.

Marking Scheme (5 Marks)

Explains linear search comparisons increase linearly	**2**
Explains binary search comparisons increase logarithmically (halving concept)	**2**

Correct comparison table/conclusion **1**

7. Describe the array representation of a queue. How are the 'front' and 'rear' pointers managed?

Answer:
A **queue** is a linear data structure that follows the **FIFO (First In First Out)** principle.

In an **array representation**, the queue is stored in a one-dimensional array.

- **Front** points to the first element of the queue.
- **Rear** points to the last inserted element.
- Initially, both front and rear are set to -1.
- During **insertion (enqueue)**:
    - If the queue is empty, set front = rear = 0.
    - Otherwise, increment rear and insert the element.
- During **deletion (dequeue)**:
    - Remove the element from front.
    - Increment front.
    - If front > rear, reset both to -1.

Marking Scheme

- Definition/FIFO – **1 Mark**
- Array representation – **1 Mark**
- Front pointer – **1 Mark**
- Rear pointer – **1 Mark**
- Enqueue/Dequeue management – **1 Mark**

8. Compare malloc(), calloc(), and realloc() with examples.

Function	Description
malloc()	Allocates a single block of memory. Memory is **not initialized**.
calloc()	Allocates multiple blocks of memory. Memory is **initialized to zero**.
realloc()	Changes the size of previously allocated memory.

Marking Scheme

- malloc() explanation – **1 Mark**
- calloc() explanation – **1 Mark**

- realloc() explanation – **1 Mark**
- Suitable examples – **2 Marks**

9.Demonstrate the use of recursion to find factorial of a number.

Answer

- Factorial of a number:
- $n! = n \times (n-1)$

```
int factorial(int n)
{
   if(n==0 || n==1)
      return 1;
   else
      return n * factorial(n-1);
}
```

Marking Scheme

- Definition of recursion – **1 Mark**
- Base case – **1 Mark**
- Recursive case – **1 Mark**
- Program – **1 Mark**
- Example/output – **1 Mark**

10. Illustrate the step-by-step changes in a stack (array representation) when the following operations are performed: push(10), push(20), pop() and push(30)

Answer:

A **stack** is a **linear data structure** that follows the **LIFO (Last In First Out)** principle. In an array implementation, the variable **TOP** keeps track of the topmost element in the stack.

Initial Condition

- Stack Size = 5
- TOP = -1 (Stack is empty)

Operation 1: push(10)

Step

- Increment TOP by 1.
- Insert 10 at index 0.

Operation 2: push(20)

Step

- Increment TOP.
- Insert 20 at index 1.

Operation 3: pop()

Step

- Remove the element at TOP.
- Decrement TOP.

Operation 4: push(30)

Step

- Increment TOP.
- Insert 30.

Marking Scheme (5 Marks)

Definition of stack and LIFO principle	**1**
Correct illustration of push(10) and updated TOP	**1**
Correct illustration of push(20) and updated TOP	**1**
Correct illustration of pop() with deleted element and updated TOP	**1**
Correct illustration of push(30) and final stack with TOP	**1**

11. Analyse the structural and functional differences between a binary tree and a binary search tree. Use an example of a binary search tree to justify your analysis.

Answer:

Binary Tree

- Each node has at most two children.
- No specific ordering of nodes.
- Searching may require checking many nodes.

Binary Search Tree (BST)

- Left subtree contains values smaller than the root.
- Right subtree contains values greater than the root.
- Searching is faster due to ordered structure.

Marking Scheme

Binary tree explanation – **1 Mark**

BST explanation – **1 Mark**

Structural differences – **1 Mark**

Functional differences – **1 Mark**

Suitable example – **1 Mark**

12. Explain how elements are shifted during insertion and deletion in a linear array.

Insertion

To insert an element at a given position:

1. Move all elements from the end towards the insertion position one place to the right.
2. Insert the new element.

Deletion

To delete an element:

1. Remove the element.
2. Shift all subsequent elements one position to the left.

Marking Scheme

- Insertion explanation – **2 Marks**
- Deletion explanation – **2 Marks**
- Suitable example – **1 Mark**

Part C Essay Questions Answer any 1 questions Each question carries 15 marks

13. Define Data Structures in C and list different types with examples.

Answer

Definition (2 Marks)

A data structure is a method of organizing, storing, and managing data in memory so that it can be accessed and modified efficiently. In C programming, data structures help perform operations such as insertion, deletion, searching, sorting, and updating data efficiently.

Classification of Data Structures (2 Marks)

Data structures are broadly classified into:

1. Primitive Data Structures
2. Non-Primitive Data Structures

1. Primitive Data Structures (4 Marks)

Primitive data structures are the basic built-in data types provided by the C language. They store a single value.

Types with Examples

int	Stores integer values	int age = 20;
float	Stores decimal values	float salary = 25000.50;
char	Stores a single character	char grade = 'A';
double	Stores large decimal values	double pi = 3.14159265;
void	Represents no value or no return type	void display();

Characteristics

- Built into the C language.
- Store a single value.
- Used as building blocks for other data structures.

2. Non-Primitive Data Structures (6 Marks)

Non-primitive data structures are derived from primitive data types and can store multiple values.

They are classified into:

A. Linear Data Structures

Elements are arranged in a sequential order.

(i) Array

- Collection of similar data items.
- Stored in contiguous memory locations.
- Fixed size.

(ii) Linked List

- Collection of nodes.
- Each node contains data and a pointer to the next node.

- Dynamic in nature.

(iii) Stack

- Follows LIFO (Last In First Out).
- Operations: Push and Pop.

(iv) Queue

- Follows FIFO (First In First Out).
- Operations: Enqueue and Dequeue.

B. Non-Linear Data Structures

Elements are not arranged sequentially.

(i) Tree

- Hierarchical structure consisting of root, parent, child, and leaf nodes.

(ii) Graph

- Collection of vertices (nodes) connected by edges.

Applications of Data Structures (1 Mark)

- Database Management Systems
- Operating Systems
- Compiler Design
- Computer Networks
- Artificial Intelligence
- File Management Systems

Marking Scheme (15 Marks)

Definition of data structure	2
Classification into Primitive and Non-Primitive	2
Primitive data structures with examples	4

Non-Primitive data structures (Linear & Non-Linear) with examples 6

Applications/Conclusion 1

14. Define Circular Queue and list its basic operations.

Answer

A **Circular Queue** is a queue in which the last position is connected back to the first position, forming a circle. It follows the **FIFO (First In First Out)** principle and efficiently utilizes memory by reusing empty spaces created after deletions.

Need for Circular Queue (2 Marks)

In a linear queue, after several dequeue operations, empty spaces at the beginning cannot be reused.

A circular queue overcomes this problem by allowing the **rear** pointer to wrap around to the beginning of the array.

Array Representation (2 Marks)

- Queue is stored in an array.
- Two pointers are maintained:
    - **Front** → points to the first element.
    - **Rear** → points to the last element.

Initially,

Front = Rear = -1

Basic Operations (6 Marks)

1. Enqueue (Insertion)

- Insert an element at the rear.
- Update rear using:

rear = (rear + 1) % MAX

2. Dequeue (Deletion)

- Remove the element from the front.
- Update front using:

front = (front + 1) % MAX

3. Peek (Front Element)

Returns the first element without removing it.

4. isEmpty()

Checks whether the queue is empty.

Condition:

front == -1

5. isFull()

Checks whether the queue is full.

Condition:

(rear + 1) % MAX == front

Advantages (2 Marks)

- Efficient memory utilization.
- Eliminates wastage of space.
- Faster insertion and deletion.
- Used in CPU scheduling, buffering, and printer queues.

Applications (1 Mark)

- CPU Scheduling
- Circular Buffers
- Printer Spooling
- Multimedia Streaming
- Traffic Signal Control

Marking Scheme

Definition of circular queue	**2**
Need for circular queue	**2**
Array representation (Front and Rear)	**2**
Basic operations (Enqueue, Dequeue, Peek, isEmpty, isFull)	**6**

Advantages **2**

Applications / Diagram & example **1**